Avocate Harvest Escrow: security audit

Soroban contracts (escrow, platform, financing pool), v0.0.4

25 October 2026

Summary
Status of the findings
Automated analysis: CoinFabrik Scout
Improvements made after the audit
Scheduled before any mainnet deployment
Appendix: manual review in detail

Summary

Two reviews of the three Soroban contracts were run on 25 October 2026, during HackMeridian, and the problems they found were fixed the same day.

- **Automated:** CoinFabrik **Scout** 0.3.17 (`cargo-scout-audit`, with `cargo-dylint` / `dylint-link` 6.2.0), on every contract, before and after the fixes.
- **Manual:** a line-by-line review against the project's own security rules and the official Stellar smart-contract security guidance (authorization trees, storage TTL and archival, checked arithmetic, cross-contract calls, upgrade gating), with proof-of-concept tests for the economic findings.

The manual review found **19 findings**: 3 high, 3 medium, 7 low and 6 informational. **Five were fixed** in code before this report (F-01, F-02, F-04, F-05, F-06) and one partly (F-16, tests), three are mitigated, and the rest are accepted or scheduled, each with its reason below. Scout's findings are patterns (arithmetic, `unwrap`, missing events); none is an exploitable path in this code, and two of them coincide with manual findings that are scheduled (F-14 events, F-19 arithmetic).

This is an internal review by the project team, not an audit by an independent security firm. It is published so judges and users can see what was checked, what was found and what changed. The contracts run on Stellar testnet only.

Contracts	`contract/contracts/escrow`, `contract/contracts/platform`, `contract/contracts/pool`
Toolchain	soroban-sdk 29.0.1 (protocol 29), Rust 1.99.1, release profile with `overflow-checks = true`
Code reviewed	commit `e58f8a0` (contracts as of v0.0.3)
Fixes	commits `fd261ab` (escrow) and `c7811f7` (pool v2), deployed in `4b32db9`
Tests after the fixes	77 passing (escrow 30, platform 17, pool 30), run in CI on every push

Status of the findings

ID	Severity	Finding	Status
F-01	High	Overdue loans are valued at zero, so a new depositor can enter at a discount and capture a later repayment	**Fixed**: deposits are paused while any advance is overdue (pool v2)
F-02	High	A rejected loan kept its full value until the buyer refunded, so depositors could exit first and leave the loss to others	**Fixed**: the escrow writes the advance off at the moment of the rejection
F-03	High	One admin key controls validator, agent, escrow code, upgrades and farmer onboarding	**Accepted for testnet**; multisig admin and separated roles are a precondition for mainnet
F-04	Medium	Early repayment paid the full yield at once, giving late depositors yield that had not accrued	**Fixed**: the farmer pays yield in proportion to time, at the same rate the pool uses to value the loan
F-05	Medium	The buyer could refund after the deadline even when the delivery had been validated as conforming	**Fixed**: no refund once the delivery conforms; the trade can only be paid out
F-06	Medium	Overdue loans could stay open forever, since only the buyer could close them	**Fixed**: anyone can trigger a deadline refund; the money still goes only to the pool and the buyer
F-07	Low	`fund` and `validate` have no deadline check	Scheduled
F-08	Low	The pool does not compare the escrow's token with its own asset	**Mitigated**: only escrows from the platform registry are financed, and the platform fixes the token; explicit check scheduled
F-09	Low	The advance cap is configurable up to 100% (default and documented value: 80%)	Scheduled: hard upper bound
F-10	Low	Each fee can be set up to 100%	Scheduled: lower caps per fee
F-11	Low	Missing accounting entries are read as zero	Scheduled
F-12	Low	Storage TTL: escrow and pool state, and registry entries, can be archived if untouched	Scheduled: TTL extension before mainnet
F-13	Low	Roles are not checked for distinctness (buyer = seller allowed)	Scheduled

ID	Severity	Finding	Status
F-14	Info	Admin and configuration changes emit no events (also reported by Scout)	Scheduled
F-15	Info	Rounding at the edges: `max_redeem` can leave one unit; redeem can burn shares for zero assets	**Accepted**: rounding always favours the pool (seen in practice: 1 base unit left after the migration)
F-16	Info	Test quality: some `should_panic` tests without an expected error; important paths untested	**Partly fixed**: 5 new tests and 2 tightened ones; more scheduled
F-17	Info	A comment on the buyer's authorization in `advance` misdescribes Soroban auth trees	Scheduled
F-18	Info	The pool credited the repayment on the escrow's word	**Mitigated**: the repayment must now fall between the principal and the full yield, and only escrows the pool lent to can report it
F-19	Info	Arithmetic on `i128` without checked operations; `unwrap()` on storage (also reported by Scout)	**Mitigated** by `overflow-checks = true` (an overflow aborts the transaction); checked arithmetic with named errors scheduled

Verified as safe by the manual review: authorization on every state-changing function, the pool's callbacks gated to the escrows it lent to, the rounding direction of all four vault operations, cash accounting that ignores direct donations, no double settlement and no double advance, the advance always fitting inside the price (`repay + financing fee <= amount`), and reentrancy.

Automated analysis: CoinFabrik Scout

Scout 0.3.17 was run on each contract before and after the fixes. It reports patterns found by its detectors; each one was triaged by hand.

Detector	Scout severity	Escrow (before / after)	Platform	Pool	Assessment
Integer overflow or underflow	Critical	8 / 9	1 / 1	21 / 21	Plain `+ - *` on `i128` . The release profile has `overflow-checks = true` , so an overflow aborts the transaction instead of producing a wrong amount. Amounts are bounded by the token supply. Not exploitable; checked arithmetic with named errors is scheduled (F-19). The escrow count rose by one with the new pro-rata formula.
Unsafe unwrap	Medium	3 / 3	14 / 14	10 / 10	`get().unwrap()` on entries written by the constructor. They can only be missing if archived, which is F-12.
Contract import dependency	Medium	–	9 / 9	14 / 14	Contract clients for cross-contract calls (platform deploys escrows; pool calls platform and escrows). Expected design.
Front running	Medium	1 / 2	–	–	Token transfers whose amounts are computed by the escrow from its fixed terms. There is no price or quote an attacker could move, so ordering cannot change the outcome. Not applicable.
DoS unbounded operation	Medium	–	2 / 2	–	The registry page loop; pages are capped at 50 entries.
Dynamic storage	Medium	–	–	3 / 3	Per-address entries (farmer limit, exposure, shares). Expected.
Storage change events	Enhancement	7 / 7	7 / 7	10 / 10	Configuration changes without events. Same as F-14; scheduled.

Scout does not model business logic, which is why the economic findings (F-01, F-02, F-04, F-05, F-06) came only from the manual review.

Improvements made after the audit

Escrow (commit `fd261ab` , new code for trades created from v0.0.4; existing trades keep theirs):

- `refund` returns `DeliveryConforming` once the validator has confirmed the delivery, before or after the deadline (F-05).
- A deadline refund no longer needs the buyer's signature, so an expired trade cannot hold the pool's money (F-06). A refund after a rejection still needs the buyer.
- `validate(false)` on a financed trade notifies the pool immediately, which writes the advance off at that moment (F-02).
- The advance records when it started; the pool is repaid the principal plus the yield for the time elapsed, capped at the full yield at the deadline, with the same linear formula the pool uses to value loans (F-04). Settling early costs the farmer less, and a repayment never moves the share price.

Pool v2 (commit `c7811f7` , deployed in `4b32db9` at `CDAHTGYCWIHSJOLUJDLUIZJOC7P7DCTTPLQLPJIIMS5ID2S6L2W7X4VK`):

- `deposit` and `mint` fail with `DepositsPaused` while any advance is past its deadline (F-01).
- `on_repaid` takes the amount actually paid and rejects anything outside principal to full yield (F-04, F-18).
- `upgrade` (admin only), so future fixes do not require another pool. The pool reviewed (v1) had no upgrade path, which is why a new pool was deployed.
- It understands the `Cancelled` state, so financing a cancelled trade fails with `NotFunded` .

Migration on testnet: the one advance open in pool v1 was validated and paid out by the agent (repaying the pool with its yield), the only depositor's balance (65.74 USDC) was redeemed from v1 and deposited into v2, the test farmer was onboarded in v2, and the platform now links v2 and the new

escrow code. A financed trade created afterwards was settled minutes after its advance: pool v2 received the principal plus a few seconds of yield, not the full 2%.

Tests added: no refund after a conforming delivery (before and after the deadline); anyone can trigger a deadline refund; a rejection refund still needs the buyer; the loss is booked at the rejection; early settlement pays pro-rata yield without a price jump; deposits pause while an advance is overdue and resume after the deadline refund; a cancelled trade cannot be financed (now with the exact error).

Documentation: the README's "Security and anti-fraud rules" were updated with each rule above, tagged with the finding it answers.

Scheduled before any mainnet deployment

Multisig admin and separate keys per role (F-03); fee and advance caps (F-09, F-10); TTL extension for long-lived entries (F-12); events for every configuration change (F-14, Scout); checked arithmetic with named errors (F-19, Scout); the remaining low findings and tests (F-07, F-08, F-11, F-13, F-16, F-17); and a review by an independent auditor.

Appendix: manual review in detail

The detail below is the manual review as written before the fixes. Line numbers refer to the code at commit `e58f8a0`; see "Status of the findings" for what changed afterwards. Amounts use whole USDC (the contracts use 7 decimals).

Findings in detail

F-01 (High): overdue loans valued at zero allow discounted entry

- **Where:** `pool/src/lib.rs` `total_assets` (L200-212): a loan with `now > deadline` contributes nothing. `to_shares` (L506-508) and therefore `deposit` (L264-273) and `preview_deposit` use that value, and `mint` uses `to_assets` (L510-512) on it too.
- **Why it matters:** the README rule "a loan past its deadline counts as zero until it is repaid, so a depositor who sees a loss coming cannot exit at a price that still includes it" protects exits. The same rule applies to entries. While a loan is overdue, the price of a share is understated, so a new depositor receives too many shares. When the escrow is later refunded, the repayment (principal plus yield) enters the pool and the new depositor keeps their share of it.
- **Exploit (PoC `poc_entrant_buys_overdue_loan_at_zero_value`, passes):** pool holds 4,000 USDC; a 1,000 USDC trade is advanced 800 USDC; the deadline passes and nothing is refunded yet. The pool's recorded value is 3,200 instead of 4,016. An entrant deposits 4,000, the buyer calls `refund` (the buyer can be the entrant, or an honest buyer may simply be slow), and the entrant redeems: **4,453.33 USDC out for 4,000 in**. The 453.33 USDC comes from the other depositors, who end at 3,563 instead of 4,016. No privileged role is needed: the entrant only has to deposit before a refund that they can see coming, and the refund can come from a passive buyer. The trade's farmer must be onboarded by the admin, as for any advance.
- **Recommendation:** choose one of: (a) refuse `deposit` / `mint` while any open loan is overdue (the loop already visits every loan); (b) value overdue loans at principal plus accrued yield for entries and at zero for exits only if you accept that asymmetry explicitly; (c) make the deadline refund permissionless (see F-06), so overdue loans close within a ledger. Option (a) is the smallest change. Note that (c) changes the README's "only the buyer can claim a refund" rule, which is a product decision.

F-02 (High): rejected loans are priced at full value until the refund, so depositors can exit first

- **Where:** `pool/src/lib.rs` `total_assets` (L200-212) counts every open loan until `on_written_off` removes it. The verdict is recorded in escrow `validate` (L290-303) but the pool is not told. The write-off happens only in escrow `refund` (L328-355, `on_written_off` at L344), and the buyer alone decides when.
- **Why it matters:** the README says that on a rejected delivery "the advance is written off. The loss is shared by all depositors in proportion to their shares." That holds only if nobody can leave between the verdict and the write-off. The verdict is public on-chain as soon as `validate(false)` is included, and redeeming needs only idle cash.
- **Exploit (PoC `poc_depositor_exits_after_rejection_is_public_before_refund`, passes):** pool holds 4,000 USDC (Alice deposited 3,000, Bob 1,000). An 800 USDC advance is made. The validator records `ok = false`. Alice redeems her full balance before the refund: **3,000 USDC out** (fair share of the eventual 3,200 is 2,400). The refund then happens and Bob can withdraw only **200 USDC** (fair share 800). The 600 USDC that Alice avoided was shifted to Bob. Any depositor watching the chain can do this, not only an insider.
- **Recommendation:** when `validate(false)` is recorded for a funded, financed escrow, let the pool mark the loan as lost immediately (a callback from `validate`, or a `pool.on_rejected` call), so `total_assets` excludes it at once. Alternatively, `total_assets` can read `escrow.validation()` for each open loan (50 cross-contract reads at most), at a higher cost per deposit and withdrawal.

F-03 (High, trust model): one admin key controls the whole financing path

- **Where:** `platform/src/lib.rs` `set_operators` (L137-143) sets the validator and the agent for every new trade; `set_escrow_wasm` (L154-158) chooses the code of every new escrow; `upgrade` (L161-164) replaces the platform code, including `is_escrow`, which the pool trusts (`pool` `advance` L337-339); `set_pool` (L147-151) links the pool. In the pool, `onboard` (L307-314) chooses the farmers and their limits, and `set_max_advance_bps` (L323-330) the advance cap. The escrow never checks that the validator is independent of the buyer or the farmer (F-13).
- **Exploit (no code defect; it is what the trust model allows):** the admin sets the validator to a key it controls and onboards a farmer it controls (or any farmer colluding with it). It creates a trade with its own buyer key, funds it, and calls `advance` for up to 80% of the price. The validator records a rejection and the buyer calls `refund`. The buyer gets the whole deposit back, including the trade fee (`refund` returns `amount + trad`

`e_fee`), so the buyer's cost is only gas. The farmer keeps the advance and the pool loses it, shared by all depositors. Losses are bounded by the per-farmer limit and by the pool's idle cash. A compromised admin key, a compromised validator key, or a validator who rejects falsely produces the same result. The pool has no check on the escrow code, the registry or the validator.

- **Why it is High:** the README describes the validator as "independent" and the loss as "shared by all depositors", but nothing on-chain enforces either. Depositors carry the risk of every off-chain decision the operator makes.
- **Recommendation:** make the admin a multisig with a timelock on `set_operators`, `set_escrow_wasm`, `upgrade` and `onboard`; emit events for each change (F-14); cap write-offs per farmer and per period; disclose in the README that the pool trusts the admin and the validator, and that they are not enforced on-chain. Enforcing distinct roles (F-13) is a cheap partial mitigation.

F-04 (Medium): early repayment hands unearned yield to late depositors (PoC)

- **Where:** escrow `settle` (L400-420) and `refund` pay the full `repay` (principal plus 2%) at any time. Pool `total_assets` (L200-212) accrues yield only with elapsed time.
- **Why it matters:** the README says "yield accrues linearly over the loan term, so nobody can deposit just before a repayment, take the yield and leave." If a trade is confirmed early, the full yield is paid at once but only the elapsed part was accrued. A depositor who enters just before the confirmation takes a share of the unaccrued part.
- **Exploit (PoC `poc_late_entrant_captures_unaccrued_yield_on_early_repayment`, passes):** 800 USDC advanced; delivery confirmed after 1/30 of the term; a 4,000 USDC depositor enters just before the repayment. The entrant gains **7.73 USDC** on the 16 USDC yield, paid by the existing depositors. The entrant's gain is about the unearned fraction times their share of the pool. Existing test `settlement_repays_the_pool_and_depositors_earn_the_yield` asserts the full 16 USDC to depositors with zero elapsed time, so the suite encodes the same behaviour.
- **Recommendation:** pay the pool principal plus yield accrued to the settlement time on early settlement (the pool already stores each loan's start time, so the formula needs no new data), or lock deposits for the loan term. The choice changes the economics, so it is for the product owner.

F-05 (Medium): buyer can refund after a conforming validation

- **Where:** escrow `refund` (L328-355): `rejected` is false for an `ok = true` verdict, so after the deadline the buyer may refund a validated trade (L332-335). `agent_release` (L308-318) requires a conforming verdict but no timing, and the two race near the deadline.
- **Why it matters:** a conforming validation gives the seller no protection once the deadline passes. If the agent is offline at the deadline, the buyer can take the money back for a delivery that was attested as conforming. The README's rule "refund … after the deadline" is consistent with this, but the README does not say that a positive verdict does not stop the buyer.
- **Recommendation:** after a positive verdict, block the deadline refund, or let `refund` be called only in the rejected case and let the agent settle at any time. This is a product decision; the contract behaves as the code says.

F-06 (Medium): overdue loans can stay open indefinitely

- **Where:** only the buyer can call `refund` (escrow L335). A loan is removed only by `on_repaid` or `on_written_off`, both called from `refund` / `settle`. The pool has a cap of 50 open loans (`MAX_OPEN_LOANS`, L21; check L374-377).
- **Why it matters:** an overdue trade whose buyer never refunds keeps its slot, keeps its principal out of the pool's value (F-01), and reduces the number of advances the pool can make. Fifty such trades stop all financing (`TooManyLoans`). The buyer loses access to their own deposit in that case, which limits the attack, but a buyer who is also the farmer can accept that cost.
- **Recommendation:** make `refund` after the deadline permissionless. The destinations are fixed (buyer and pool), so no caller authority is needed; this is a rule change, see F-01.

F-07 (Low): no deadline check in `fund` and `validate`

- **Where:** escrow `fund` (L209-224) has no check against `deadline`; `validate` (L290-303) neither.
- **Impact:** a buyer can fund an expired trade. The money can only be refunded (immediately, because the deadline has passed) or settled. No loss to others, because the pool refuses to finance it (`advance` L349-351), but the trade is funded and useless. Add `if now >= deadline` checks to `fund` if a funded trade should never start after its deadline.

F-08 (Low): the pool does not check the escrow token against its own asset

- **Where:** pool `advance` (L334-401) transfers the pool asset to the farmer. Escrow `settle` and `refund` pay the escrow's `token` (taken from the platform's `token`, which is set only in the constructor). `on_repaid` (L404-410) credits `loan.repay` to the pool's cash without checking what token arrived.
- **Impact:** if the platform was deployed with a token other than the pool's asset, the pool would count repayments it never received, and cash would be overstated. Only a deployment error can cause this. The check is one line: `config.token == query_asset(&env)` in `advance`.

F-09 (Low): advance cap configurable up to 100%

- **Where:** pool `set_max_advance_bps` (L323-330) allows any value up to `BPS_DENOMINATOR`. The README says "up to 80% of the price".
- **Impact:** the escrow still requires `repay + financing fee <= price` (L271-274), so the total cannot exceed the price, but the documented 80% limit can be raised by the admin without notice. Either document the admin power or bound the value.

F-10 (Low): fee bounds allow 100% per fee

- **Where:** escrow constructor (L182-185) and platform `check_fees` (L297-302) accept any value up to 10,000 bps for each fee.
- **Impact:** a 100% trade fee doubles the buyer's payment (the buyer must lock `amount + fee`). Pool yield and financing fee at 100% are bounded by the price check in F-09's `register_advance`. A lower sanity cap (for example 10%) would stop a mistyped fee from charging the buyer double. The value is a product decision.

F-11 (Low): missing accounting entries read as zero

- **Where:** pool `exposure_of` (L458-460), `shares_of` (L555-557) use `unwrap_or(0)` on persistent entries. `take_loan` (L449-450) computes `exposure_of(...) - loan.principal` and writes it back with `set_exposure`, which stores a negative value if the entry is missing.
- **Impact:** with the current Soroban behaviour (archived entries are restored automatically when a transaction declares them, from protocol 23 on), an exposure entry is not lost. If an entry were ever reported as absent during `on_repaid`, the exposure would become negative and the per-farmer cap would be bypassed for that farmer. I did not verify how the host reports an archived but not-restored entry to `get`; this needs a ledger-snapshot test. Recommendation: in `take_loan`, treat a missing exposure as an error, not zero; and never store a negative exposure (`checked_sub` with a panic).

F-12 (Low): TTL and archival

- **Escrow:** the instance TTL is extended only on writes, to 30 days (`bump` L433-435). A funded trade with no transaction for more than 30 days is archived with its funds. The funds are not lost (archived state is restorable), but every entry point then needs a restore first.
- **Pool:** the instance (cash, loans, total shares, admin) is extended to 120 days on each write (`bump` L569-571). If no pool transaction happens for 120 days, the whole pool is archived. A restore is needed before any call. Withdrawals then need the restore too.
- **Platform registry:** `EscrowIndex` and `Escrow(i)` are persistent entries extended only at creation (L285-290). After about 120 days without a restore, `is_escrow` may return false for a live trade, so a long-dated trade can no longer be financed until the entry is restored.
- **Mitigations already in place:** the contracts do not use TTL as a security mechanism, which is correct because anyone can extend any TTL. The README is silent on TTL, which should change for operators: the deploy scripts should bump instances periodically, and the registry entries should be refreshed when an escrow is touched (`is_escrow` is not a write, so this needs a keeper).

F-13 (Low): role distinctness is not enforced

- **Where:** `create_escrow` (L169-178) and `create_trade` (L182-192) accept any buyer and seller, and the escrow constructor (L178-206) accepts any validator and agent, including equal addresses.
- **Impact:** a buyer who is also the seller can self-finance a trade (F-03). Validator equal to buyer or seller removes the independent check. This is a cheap guard: reject `buyer == seller` and `validator ∈ {buyer, seller}`.

F-14 (Informational): missing events

- **Where:** platform `set_treasury` (L129-133), `set_operators` (L137-143), `set_pool` (L147-151), `set_escrow_wasm` (L154-158), `upgrade` (L161-164); pool `onboard` (L307-314), `offboard` (L317-321), `set_max_advance_bps` (L323-330). `set_fees` emits `FeesUpdated`, so the others are inconsistent.
- **Impact:** off-chain monitoring cannot see changes to the operators, the pool link, the escrow code, the upgrade or farmer limits. Recommended for F-03.

F-15 (Informational): rounding at the edges

- `redeem` (L294-303) uses `to_assets(..., Down)` without a minimum, so a tiny share amount burns shares for zero assets. The depositor loses only their dust; nobody else gains.
- `max_redeem` (L255-258) rounds down. In the PoC, the last depositor's full balance was one unit of rounding above `max_redeem`, and `redeem(balance)` reverted with `ExceedsMax`. Callers should use `max_redeem`. The +1 in `total_assets + 1` and the virtual shares are the intended mitigation (see below), not a bug.

F-16 (Informational): tests

- `should_panic` without an expected error code (so any panic passes): escrow `test.rs` L270 and L280, pool `test.rs` L447 and L457 (`a_cancelled_trade_cannot_be_financed` could pass for an unrelated panic), platform `test.rs` L121. Use `expected = "Error(Contract, #N)"` for each.
- Untested paths (no test name exercises them): `TooManyLoans` (50-loan cap), `InsufficientCash`, `UnknownLoan` (on_repaid from an unknown escrow), `WrongPool`, `NotFunded`, `AdvanceTooLarge`, `AlreadyAdvanced`, `set_max_advance_bps` (and its bound), `upgrade`, `set_treasury`, `offboard` with an open loan (does take_loan still work), early repayment before the term ends (F-04 is encoded as expected behaviour), overdue loan refunded by the buyer with an `ok = true` verdict (F-05), `fund` after the deadline (F-07), an escrow whose advance later has its exposure entry restored (F-11), and any path that reaches the archived-state behaviour (F-12). The README's "covered by tests" statement is therefore true only for the items it lists.

F-17 (Informational): comment on auth in `advance`

- Pool `advance` L354-357 says that Soroban "only accepts an authorization rooted in the top-level call" and that the nested check "joins that tree". The Stellar docs describe authorization as a tree that the signer authorizes, including sub-invocations, not as a rule that only the top-level

call counts. The duplicate `config.buyer.require_auth()` is still correct defence in depth (the buyer's signature is bound to the exact `advance` arguments, and the escrow's own check then applies to the same tree). Fix the comment so the next reader does not rely on the wrong model.

F-18 (Informational): repayment is trusted, not verified

- Pool `on_repaid` (L404-410) credits `loan.repay` without verifying the token movement. Escrow `settle` and `refund` transfer before the callback (L346-347, L408-409), so the Avocate escrow code is correct. The guarantee rests on escrow code being the Avocate one, which rests on the admin (F-03).

F-19 (Informational): arithmetic and panics

- `mul_div` (L518-524) uses `checked_mul(...).expect("overflow")`. Overflow aborts the transaction (no state change), and the release profile has `overflow-checks = true` in any case. Test profile overflows panic as well.
- Accrual in `total_assets` (L205-207) is floored, so a loan's accrued value is at most one unit low per evaluation. This favours new entrants by a negligible amount.
- `unwrap()` on storage (pool L193, L363, L434, L446, L455, L473, L477, L540, L548) is safe because the constructors write every key before any other call; `open.get(index)` at L446 follows a successful `position` lookup. Escrow `state()` (L358) and `config()` (L380) likewise.
- Escrow `bps` (L427-429): `amount * rate / 10000` with `i128` and checked overflow; amounts far beyond realistic USDC values would abort, not corrupt.

Checks requested

- **Authorization on state-changing functions.** Escrow: `fund` (buyer), `cancel` (named party, `by.require_auth()`, buyer or seller only), `confirm_delivery` (buyer), `register_advance` (pool and buyer), `validate` (validator), `agent_release` (agent plus validation), `refund` (buyer). Pool: `deposit` / `mint` / `withdraw` / `redeem` (owner = operator, enforced by `require_owner`), `onboard` / `offboard` / `set_max_advance_bps` (admin), `advance` (farmer and buyer), `on_repaid` and `on_written_off` (the lent escrow itself, plus a loan lookup). Platform: every setter and `upgrade` (admin); `create_escrow` (buyer); `create_trade` (buyer and seller). No state-changing function is missing its check.
- **Who can call pool callbacks.** Only the escrow address recorded in the loan can call them, because `escrow.require_auth()` must succeed for that address and `take_loan` finds the loan by escrow address. A non-lent escrow fails with `UnknownLoan`. Residual trust: the Avocate escrow code must transfer the repayment (F-18).
- **Rounding direction.** `deposit`: shares Down (favours pool). `mint`: assets Up. `withdraw`: shares Up. `redeem`: assets Down. The matching previews agree. `convert_to_*` use Down for both directions, which is a view and not a transfer. `max_withdraw` and `max_redeem` are capped by idle cash, and I showed that `withdraw` cannot underflow the owner's shares (`to_shares(to_assets_down(s), up) <= s`).
- **Share price manipulation by donations.** Cash is internal (`Cash` key), so direct transfers do not change the price. The virtual offset (`10^3` shares, `+1` asset) prevents the first-depositor inflation attack: the first depositor cannot make a later deposit round to zero. Price changes come only from loan accrual, repayments and write-offs, which are the intended economics. The remaining manipulation vectors are F-01, F-02 and F-04.
- **Overdue and write-off accounting.** Overdue loans count as zero (intended for exits, but see F-01). Write-off removes principal and exposure (`take_loan` L449-450). Repayment adds the full `repay`. Exposure is reduced on both paths. Overdue loans are not refunded automatically (F-06).
- **Can an advance exceed what settle or refund can pay?** No. `register_advance` enforces `repay + bps(principal, financing_fee) <= amount` (escrow L271-274). Settlement pays `repay` to the pool and `amount - repay - financing_fee` to the seller and `trade_fee + financing_fee` to the treasury, which sums to `amount + trade_fee`. Deadline refund pays `repay` to the pool and `amount + trade_fee - repay` to the buyer. Both paths conserve the buyer's deposit (I checked the arithmetic line by line; no test covers the overflow boundary).
- **Can a trade be settled twice?** No. `require_state` (Funded) runs first, and `set_state` (Released or Refunded) runs before any transfer. A failed transfer reverts the state change. A second `advance` is rejected by `AlreadyAdvanced`.
- **TTL expiry risks.** See F-12. The registry is persistent and extended only at creation; the loans, cash and shares depend on the pool's instance TTL. Anyone can extend TTLs (`ExtendFootprintTTLOp`), so nothing in the contracts depends on TTL for security, which is correct.
- **Loans cap and total_assets cost.** At most 50 loans, each a small struct; `total_assets` is O(50) and runs once per `to_shares` / `to_assets` call, which is a constant number of calls per operation. Gas is fine. The cap itself is also a DoS vector for financing (F-06).
- **Admin key centralisation, fee changes, upgrade.** See F-03. Fee changes apply to new trades only (snapshotted in `open_trade` L269-276). The platform is upgradeable by the admin (no timelock, no migration guard); the pool and escrow are not (checked: no `update_current_contract` in either). Changing the escrow wasm affects only future trades but is equivalent to controlling their terms.
- **Fee bounds.** See F-10. 100% is not reasonable as a default bound; a lower sanity cap is a product decision.
- **cancel() edge cases.** Cancel works only in Pending. Once funded (even after the deadline) it cannot be cancelled, and after an advance there is no cancel path. A seller can cancel a pending trade before the buyer funds it: this is the README's stated design, not a bug (I-level observation).
- **create_trade auth.** Both `buyer.require_auth()` and `seller.require_auth()` are called in the same transaction (platform L189-191). The two-party signature requirement holds.
- **Events.** Covered: escrow `Funded`, `Released`, `Advanced`, `Validated`, `Refunded`, `Cancelled`; pool `Advanced`, `Repaid`, `WrittenOff`, `Deposit`, `Withdraw`; platform `EscrowCreated`, `FeesUpdated`. Missing: see F-14. The escrow constructor emits nothing; the platform's `EscrowCreated` is the registry event.
- **Panics versus errors.** Business rules use `Error` codes (escrow, pool, platform), which is good for clients. `expect("overflow")` (F-19) and `unwrap()` on storage are the only panic paths with string or no code; both are safe here.

- **Reentrancy and cross-contract calls.** Soroban blocks direct and indirect reentry, and the code updates state before transfers and callbacks (escrow `set_state` before transfer; pool `set_cash` after the callback's own state). The pool calls the escrow's `register_advance`, which does not call back. The escrow calls `on_repaid` after transferring. No reentrancy issue found.
- **Front-running.** Cancel and fund race cleanly (only one succeeds). Refund and agent release race near the deadline (F-05). Deposit and repayment timing is the subject of F-01, F-02 and F-04. Advance races are closed by `AlreadyAdvanced` and the exposure check.

Invariants from the README

Status: **Verified** = enforced in code and covered by a test or by direct reading; **Verified (caveat)** = enforced, with a limit stated; **Not verified** = not enforced, or contradicted by a PoC.

Trade escrow

Rule (README)	Status	Why
Trade terms fixed at creation; nobody can redirect the payout or change the amount	Verified	`Config` is written once in the constructor; no setter; the escrow has no upgrade function. Caveat: the platform admin can change `escrow_wasm` for future trades only (F-03).
Only the buyer can fund, confirm or claim a refund	Verified (caveat)	`fund`, `confirm_delivery`, `refund` require the buyer. `cancel` allows the seller too, as the README states for the Pending state.
The validator attests a delivery once	Verified	`validate` rejects a second call (`AlreadyValidated`); tested.
The agent can only trigger the agreed payout, and only after a conforming validation; it cannot choose recipients or amounts	Verified	`agent_release` requires `ok == true`; `settle` pays fixed addresses from `Config`.
A refund before the deadline needs a non-conforming validation from the independent validator	Verified (caveat)	`refund` checks `rejected` or the deadline. "Independent" is not enforced: the validator is an admin-chosen address and roles are not distinct (F-03, F-13).
State changes before any token moves, so a trade can never be settled twice	Verified	`require_state` then `set_state` before every transfer; tested (`a_refunded_escrow_cannot_be_settled`, `a_released_escrow_cannot_be_refunded`).
Before any money is deposited, buyer or seller can cancel (state `Cancelled`); once funded it ends only by settlement or refund	Verified	`cancel` requires Pending and a named party. Tested.

Platform

Rule (README)	Status	Why
Fee schedule, service accounts and escrow code are admin-only; changes apply to new trades only	Verified (caveat)	All setters call `admin.require_auth()`; terms are snapshotted at creation. Caveat: no timelock; `upgrade` can change the platform's registry behaviour (F-03).
Only escrows deployed by the platform are listed; `is_escrow` exposes the check	Verified (caveat)	`EscrowIndex` is written only in `open_trade`. Caveat: TTL of registry entries (F-12) and admin upgrade (F-03).
No trade can be opened until the admin links the financing pool	Verified (caveat)	`open_trade` returns `PoolNotSet`; `set_pool` does not check the address (I-level).
A trade agreed on the spot (`create_trade`) needs both buyer and farmer signatures in the same transaction	Verified	`buyer.require_auth()` and `seller.require_auth()` in `create_trade`; tested.

Financing pool

Rule (README)	Status	Why
Idle cash is tracked internally; sending USDC straight to the pool cannot move the share price	Verified	`Cash` key; tested (`a_direct_transfer_does_not_move_the_share_price`).

Rule (README)	Status	Why
Virtual shares and assets make the first-depositor inflation attack unprofitable; a deposit too small to mint a share is rejected	Verified	Offset `10^3`, `+1` asset; `ZeroShares` on `deposit`; tested. Note `mint` of a dust amount is not rejected, but it charges at least one unit.
Rounding always favours the pool, so repeated small operations cannot drain the other depositors	Verified	Directions in the table under “Checks requested”; F-15 and the accrual floor (F-19) are negligible.
Shares are not transferable and there are no allowances; only the owner can deposit their funds or redeem their shares	Verified	No transfer or approve function; `require_owner` on every entry.
Only idle cash can be withdrawn; money lent to farmers stays until repaid	Verified	`max_withdraw` and `withdraw` use `min(..., cash)`; `redeem` checks `max_redeem`.

Financing (advances)

Rule (README)	Status	Why
Only genuine Avocate trades can be financed: registry check, the escrow names this pool, it is funded, and within its deadline	Verified (caveat)	`is_escrow`, `WrongPool`, `NotFunded`, `Expired`; tested. Caveat: the registry is admin-controlled through upgrade (F-03). Note `fund` has no deadline check, so an expired funded trade can only be refunded (F-07).
Only farmers onboarded by the admin get advances, up to 80% of the price and a per-farmer cap; one advance per trade; none once a verdict exists	Verified (caveat)	`FarmerNotOnboarded`, `AdvanceOverLimit`, `ExposureOverLimit`, `AlreadyAdvanced`, and the `validation` check in `register_advance`. Caveat: the 80% is a default the admin can raise to 100% (F-09).
Both farmer and buyer sign the advance; the buyer accepts that a deadline refund repays the pool first	Verified	`farmer.require_auth()` and `buyer.require_auth()` in pool `advance`; `register_advance` checks the pool and buyer again.

Rule (README)	Status	Why
The amount owed is computed by the escrow from its snapshotted terms, never taken from the caller	Verified (caveat)	`register_advance` takes only `principal` and computes `repay` from `pool_yield_bps`. The pool computes the same value from the same terms (identical formula, tested by the 816 USDC example).
Deadline refund on a financed trade: pool repaid first, buyer gets the rest; colluding gains nothing	Verified	`refund` repays `repay` before the buyer transfer; tested (`deadline_refund_repays_the_pool_first_so_colluding_gains_nothing`).
Rejected delivery: buyer gets the whole deposit back and the advance is written off; loss shared by depositors in proportion to shares	Verified (buyer side); **Not verified (depositor side)**	The buyer gets the whole deposit (tested). The loss is not shared proportionally when a depositor exits between the verdict and the refund (F-02, PoC C).
Yield accrues linearly, so nobody can deposit just before a repayment, take the yield and leave	**Not verified**	Linear accrual is enforced, but repayment pays the full yield at once, so an early repayment lets a late depositor capture unearned yield (F-04, PoC B). The existing settlement test encodes the full-yield behaviour.

Rule (README)	Status	Why
A loan past its deadline counts as zero until repaid, so a depositor who sees a loss coming cannot exit at a price that still counts it	Verified (exit side)	`total_assets` skips overdue loans (L204). The same rule lets a new depositor enter at a discount (F-01, PoC A).
The pool only accepts repayment and loss notices from escrows it actually lent to	Verified	`escrow.require_auth()` plus `take_loan` lookup; `UnknownLoan` otherwise. Tested (`only_a_lent_escrow_can_report_a_repayment`).

Out of scope for this report

Web app rules (signatures in the user's wallet, the service routes, x402 payments, Soroswap XLM funding, link and QR handling) are not part of the three contracts and were not audited.

Other observations (not README rules)

- The escrow's `Parties.platform` field holds the treasury address, not the platform contract. The name is confusing for integrators; rename it in a future version (documentation only).
- `Escrow::advance()` (view) and the `Advance` struct share names; harmless.
- The upgrade surface is limited to the platform. Pool and escrow code are immutable, which removes one class of risk but means a bug in either requires a new deployment and migration.
- Not verified: whether the USDC issuer can apply clawback to the contract balances of the escrow or pool. This depends on the asset's flags and was not checked.
- Not verified: how the host reports an archived but not-restored persistent entry to `get` (F-11). Needs a ledger-snapshot test.